PYTHON VARIABLE

A variable is a container that holds information (data)

name = "Baby"

name → container (variable)

"Baby"→ information stored

it's a name given to a memory location which holds the actual value.

A variable can holds objects of different types

All variable should be in lower case as there are more than one word in the variable then user separates with underscore and this

is called Snake_case.

syntax:--> var_name=value

Example

Prabhu Gouda

Multiple Variable Assignment

Same Value to Multiple Variables

```
a = b = c = 100
```

Multiple Values to Multiple Variables

```
x, y, z = 1, 2, 3
```

Swapping Variables (No Temp Variable)

```
a = 10
b = 20

a, b = b, a
```

Variable Memory (id & reference)

```
x = 100
y = x
print(id(x))
print(id(y))
```

Both point to **same memory**

Prabhu Gouda

Extra Information

Snake_case

snake_case is a naming style where

● words are written in lowercase

● words are separated using underscore (_)

Example

student_name = "Rahul"

total_marks = 450

🐫 camelCase

camelCase is a naming style where

● the first word starts with lowercase

● each next word starts with a capital letter

● no spaces or underscores

Example

studentName = "Rahul"

Prabhu Gouda

totalMarks = 450

🏛️ PascalCase

PascalCase is a naming style where

- every word starts with a capital letter
- no spaces or underscores

example

StudentName = "Rahul"

TotalMarks = 450

type()

type() is used to find the data type of a variable or value.

```
x = 10
y = "Python"
print(type(x))
print(type(y))
```

id()

id() is used to find the memory address (identity) of

Prabhu Gouda

an object. And constant id will be (-5 to 256)

```
a = 10
b = 10
print(id(a))
print(id(b))
```

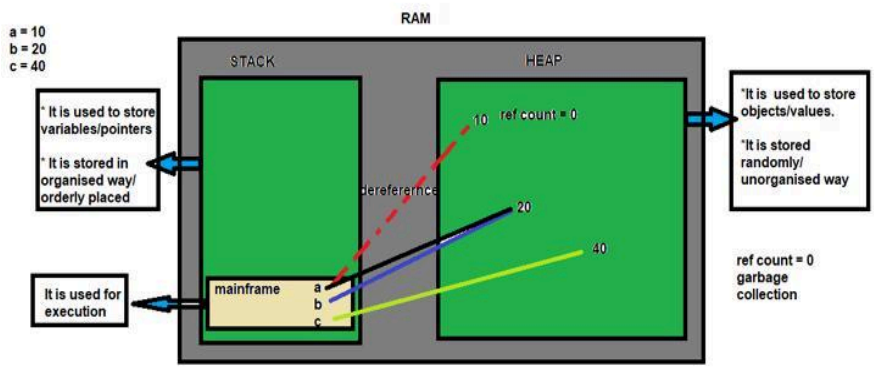
Memory management in variable

Explanation see the attached diagram

interview question

Prabhu Gouda

MEMORY MANAGEMENT IN VARIABLES.



reference count of 10 is 0
reference count of 20 is 2
referenec count of 40 is 1

THEORY : When we want to execute any code in Python RAM is required.

* RAM stand for Random Access Memory
*RAM is divides into 2 parts
> STACK
> HEAP
*STACK : It is used to store variables in Organised manner
*HEAP : It is used to store values/ojects in Randomly
*mainframe : It is used for execution

a = 10
b = 20
c = 40

Variables/Pointers starts pointing towards respective values/objects, and the reference count of each value will be 1.
Since python supports dynamic memory alloction and we can change the value for variables whenever we required, we will deference 10 from variable 'a' and starts pointing towards variable 'b'. Now, the reference count of 20 becomes 2 and 10 becomes 0, because no variable pointing towards 10.
When the reference count becomes zero(0) the garbage collection comes into picture and remove the valule from memory and free up the space and this is how memory managemnet happens in Variables.

DATA TYPE

----->IT is a type of data which we are storing inside a variable

---> all the variable are having some data.and all the data have its data type
--->no need to mention the type of the data because its is a Dynamically Typed language

*Data Type Classified into 2 Types

1.individual data type/single value data type

2.collection data type/multi value data type

Individual data type:------>

A value that is stored inside a single memory block is known as Individual data Type/single value data type

a=10

b=True

c=False

d=5.6

e=5+6j

COLLECTION DATA TYPE:----->

Prabhu Gouda

Group of charcater.group of element is called as collection

collection data type we have 5 type

1.string data type

2.list data type

3.Tuple data type

4.set data type

5.dictionary data type

#Note:---> all collection data type we can call it as a iterable

STRING DATA TYPE

it is a sequence of character it will enclosed with single,double,triple quotes is called as string data type

Boundary condition of string data type is ------->" "

Prabhu Gouda

#string data type we can represent as a str

string data type it will accept duplicate characters

1.How to check the length of the string----->?

by using len( ) function

syntax:--> len(var_name) or len(string)

2.what is the meaning of len function---->??

it will return number of characters present inside the string and it will consider space also

3.what is the syntax of string data type---->?

var_name="string"

#How to create the empty string data type---->??

var_name=" "

or

Prabhu Gowda

var_name=' '

or
 var_name=''' '''

or

var_name=""" """

How to create empty string by using constructor

var_name=str( )

Indexing:----->

The process of fetching single single characters in a string is called as indexing

indexing we can do it in two ways forward and backward direction

forward direction(left to right) always start from ---->0

backward direction(right to left) always start from----->1

Prabhu Gouda

syntax:-->

var_name[indexpostion/number]

note:---> if the specified position is not present it will throw index error

```
h="python programming"

h[2]

't'
h[6]

' '
h[-3]

'i'
h[-6]

'a'

h[100]

Traceback (most recent call last):
  File "<pyshell#259>", line 1, in <module>
    h[100]
IndexError: string index out of range
```

Prabhu Gouda

#slicing:----->
The process of fetching multiple characters is called as slicing

* in slicing we have three types

1.positive slicing

2.Negative slicing

3.combination (+ve-ve,-ve+ve)

Positive slicing syntax is

var_name[start index:end index+1:stepvalue]

start index:---> by default always 0

end index:---> by default it will take length of the string

stepvalue:---> by default it will one

Negative slicing syntax is

var_name[start index:end index+1:stepvalue]

Prachi Gouda

```
startindex:---> -1

end index:--->-len(string)

stepvalue:-->-1


Note:--> in backward slicing stepvalue is
mandatory

Note2:--> in forward slicing step value by
default it will take 1
```

example:-->

```
x="be active in class"

#active

#in

#lass

#be

#atv

s=”hey guys you all are Osam”

#Osam
```

Prabhu Gouda

#guys

#all

#hey

For finding all the attributes present inside a data type
we have to use dir( )

syntax:---> dir(data type)

string methods

****

1.upper( )

It will convert all the lowercase to upper case

syntax:--> var_name.upper( ) or string.upper( )

2.lower( )
It will convert all the upper case characters into lower case
syntax:---> var_name.lower( ) or string.lower( )

Prabhu Gowda

3.swapcase( )
It will swap the characters(upper to lower and lower to upper)

syntax:--> var_name.swapcase( ) or string.swapcase( )

4.count( )

it will give the count of a specified character/substring inside a string

if the value is not present it will return--->0

syntax :---> var_name.count("substring",[si][ei])

5.capitalize( )

convert 1st characters/letter of a sentence to uppercase followed by lower case

syntax:--> var_name.capitalize( )

Prachi Gouda

6.title( )

convert first character/letter of a word and after the space any word present means it will convert
into upper case

syntax:-->var_name.title( )

Examples:----->

```
#upper:--->
#syntax :---> var_name.upper()

a="hello"
a.upper()
'HELLO'



#lower()

#syntax:---> var_name.lower()

b="bye"
b.lower()
'bye'

c="BYE"
c.lower()
```

Prabhu Gouda

```
'bye'



#swapcase()

#syntax:--> var_name.swapcase()

d="ABCabc"
d.swapcase()
'abcABC'




7#count():--->
#syntax:---> var_name.count("sustring",[si][ei])

x="good morning"

x.count("o")
3
x.count("n")
2

x.count("o",4,7)
1

x.count("o",0,4)
2
```

Prabhu Gouda

```
x.count("z")
0

#capitalize

a="good day"
a.capitalize()
'Goodday'



#title()

a="hi good night"
a.title()
'Hi Good Night'

8.replace()

#it is used to replace characters
#syntax:-->var_name.replace("oldvalue","newvalue"
,[count])

a="yellow"
a.replace("l","*")
'ye**ow'

a.replace("l","*",1)
'ye*low'
```

Prathu Gouda

Immutable data type

Here we can’t change the original object but We can do the modification

Example

String data type
Tuple data type
a=’PYTHON” #original object

a.lower() #Modification
python

Again if we call variable names the output is always original one only not updated one reason string data type is immutable data type.

Prabhu Gouda

9.isalnum( )

It is a combination of alphabets +numbers

It returns True if the string is alphanumeric/alphabets/numbers

syntax:---> string.isalnum( ) or var_name.isalnum( )

10.isalpha( )

It will return True if all the characters in a string are letter (upper/lower),otherwise False

syntax:---> string.isalpha( ) or var_name.isalpha()

11.isdigit( )

It will return True if all the characters in a string are number/digit,otherwise False

Prabhu Gouda

syntax:--> string.isdigit( ) or var_name.isdigit( )

12.is upper( )
It will return True if all the letters in a string are uppercase,otherwise False

it doesn't care about number and special characters
syntax:--> string.isupper( ) or var_name.isupper( )

13.islower( )
It will return True if all the letters in a string are lower case,otherwise False
syntax:--> string.islower( ) or var_name.islower( )

14.isspace( )
It will return True if all the characters in a string are space/empty,otherwise False

syntax:---> string.isspace( ) or var_name.isspace( )

example:-->

```
#isalnum():---> combination of alphabets+numbers

#syntax:---> var_name.isalnum()

a="GOOD"
a.isalnum()
True

b="good"
b.isalnum()
True

c="1234"
c.isalnum()
True

d="GOODgood1234"
d.isalnum()
True

e="abc@"
e.isalnum()
False
```

Prabhu Gouda

```
#isalpha():----> here we have to pass only
alphabets

a="asdfghjkl"
a.isalpha()
True

b="ASDFGH"
b.isalpha()
True

c="2asdfg"
c.isalpha()
False

d="@#$%dfgh"
d.isalpha()
False
```

```
#isdigit():--->

a="123"
a.isdigit()
True

b="asdfgh123456"
b.isdigit()
False
```

Prabhu Gouda

```
#islower()
s="asdfgh"
s.islower()
True

v="asdfD"
v.islower()
False



#isupper()
d="ASDFG"
d.isupper()
True

b="asdfghASDFGH"
b.isupper()
False



#ispace()

s="    hello"
s.isspace()
False
```

Prabhu Gouda

```
b=" "
b.isspace()
True
```

15.startswith( )

It will return True if the string is starting with specified characters,otherwise False

syntax:-->

```
string.startswith("substring",[si],[ei]) or
var_name.startswith("substring",[si][ei])
```

16.endswith( )

It will return True if the string is ending with the specified characters,otherwise False

syntax:--> `string.endswith("substring",[si][ei])` or `var_name.endswith("substring",[si][ei])`

examples:-->

startswith():--->

Prabhu Gowda

```
#syntax:--->
var_name.startswith("substring",[si][ei])

a="good day gone"
a.startswith("good")
True

a.startswith("ood")
False

a.startswith("g")
True

a.startswith("ood")
False
a.startswith("ood",1)
True

a.startswith("day",5)
True
```

Prabhu Gouda

```
#endswith()
#syntax:--->
var_name.endswith("substring",[si][ei])

a="good day gone"
a.endswith("gone")
True
```

```
a.endswith("day")
False

a.endswith("day",0,8)
True

a.endswith("good"0,4)
SyntaxError: invalid syntax. Perhaps you forgot a
comma?
a.endswith("good",0,4)
True
```

17.join():------>

```
#iterable:--->all the collection data types are
iterables
(str,list,tuple,set,dict)

#joins the elements of an iterable using the
string specifiers

#syntax:---->

"string".join(iterable)
```

Prachu Gouda

```
#example:------>


#join()------>

a="good morning"
"-".join(a)
'g-o-o-d- -m-o-r-n-i-n-g'

"-".join("good morning")
'g-o-o-d- -m-o-r-n-i-n-g'

"@#".join(a)
'g@#o@#o@#d@# @#m@#o@#r@#n@#i@#n@#g'

"55667".join(a)
'g55667o55667o55667d55667
55667m55667o55667r55667n55667i55667n55667g'
```

18.strip( )

It removes any leading and trailing characters(space is the default characters to strip)

syntax:--> var_name.strip(characters)

19.rstrip( )

Prabhat Gouda

it removes any trailing characters

syntax:-->var_name.rstrip(characters)

20.lstrip( )
it removes any leading characters

syntax:-->var_name.lstrip(characters)

```
s="#@hello#@"

s.strip("@")
'#@hello#'

s.strip("#@")
'hello'

s.strip("#@")
'hello'

s.lstrip("#@")
'hello#@'

s.rstrip("#@")
'#@hello'
```

21.index( )

Prabhu Gouda

it will return the position/index of a specified value

it will return value error if the specified value is not present

syntax:--> var_name.index("substring",[si][ei])

22.rindex( )

it will return the position/index of a specified value

it will return value error if the specified value is not present

syntax:--> var_name.rindex("substring",[si][ei])

23.find( )

it will return the position/index of a specified value

it will return -1 if the specified value is not present

Prachi Couda

syntax:--> var_name.find("substring",[si][ei])

24.rfind( )

it will return the position/index of a specified value
it will return -1 if the specified value is not present

syntax:--> var_name.rfind("substring",[si][ei])

#index():---->
#syntax:--->var_name.index("sustring",[si][ei])

```
y="good Night"

y.index("N")
5

y.index("o")
1

y.index("g")
0

y.index("g",1)
```

Prabhu Gowda

```
7
y.index("g",5)
7

y.index("x")
Traceback (most recent call last):
  File "<pyshell#151>", line 1, in <module>
    y.index("x")
ValueError: substring not found



#rindex()
#syntax:--->var_name.rindex("sustring",[si][ei])

y="good Night"
y.index("o")
1

y.rindex("o")
2

y.index("g")
0

y.rindex("g")
7

y.rindex("b")
Traceback (most recent call last):
  File "<pyshell#165>", line 1, in <module>
```

Prabhu Gouda

```
    y.rindex("b")
ValueError: substring not found
```

```
#find()
#syntax:--->var_name.find("sustring",[si][ei])

y="good Night"

y.index("g")
0

y.find("g")
0

y.find("x")
-1

y.rfind("g")
7
y.find("g")
0
```

25.split():----------->

--> it splits the strings at the specified separator and returns a list---->[]

-->by default split at space

Prabhu Gouda

#syntax:---->

```
variable_name.split("separator"/Delimiter,maxspli
t)
```

26.rsplit():---->
it splits the strings at the specified separator
and returns a list--->[]

by default split at space

#syntax---->
```
variable_name.rsplit("separator",maxsplit)
```

#note:---->

--> if you don't specify the max split in rsplit
the output
will look the same as the split method.

--->converts a string into a list

#split( )-------->
```
a="good afternoon guys"
a.split( )
```

Prachu Gouda

```
['good', 'afternoon', 'guys']

b="goodmorning good monday"
a.split("m")

['good afternoon guys']
b.split("m")
['good', 'orning good ', 'onday']

b.split("m",1)
['good', 'orning good monday']

c="python is a programming language"

c.split("o")
['pyth', 'n is a pr', 'gramming language']

c.split("o",2)
['pyth', 'n is a pr', 'gramming language']

c.split("o",1)
['pyth', 'n is a programming language']

c.split("m")
['python is a progra', '', 'ing language']

c.split("m",1)
['python is a progra', 'ming language']

#rsplit()------>
c="python is a programming language"
```

Prabhu Gouda

```
c.split("o",1)

['pyth', 'n is a programming language']
c.rsplit("o",1)

['python is a pr', 'gramming language']
c.rsplit("m",1)
['python is a program', 'ing language']

c.split("o")
['pyth', 'n is a pr', 'gramming language']

c.rsplit("o")
['pyth', 'n is a pr', 'gramming language']
```

27.removeprefix()

removeprefix() removes a **specific prefix** (starting part) from a string **only if it exists.** If the string does **not** start with the given prefix, the **original string is returned unchanged.**

Syntax ➖

```
var_name.removeprefix("character")
```

Prabhu Gouda

```
s = "HelloWorld"
print(s.removeprefix("Hello")) #World


print(s.removeprefix("llo")) #"HelloWorld"
```

28.removesuffix()

removesuffix() removes a **specific suffix** (ending part) from a string **only if it exists.**
 If the string does **not** end with the given suffix, the **original string is returned unchanged.**

Syntax

var_name.removesuffix("character")

```
s = "filename.txt"

print(s.removesuffix(".txt"))


mail = "student@gmail.com"

print(email.removesuffix("@gmail.com"))
```

Prabhu Gupta

#interview questions

1.what is the difference between find and index method

2 what is the difference between indexing and slicing

4.what is the meaning of mutable data type with example

5.why we are using triple quotes in string data type

6.what is the difference between split( ) method and rsplit( )

7.what is the use of dir ( ) function

8.how to create an empty string using object----->?

TUPLE DATA TYPE

Prabhu Gowda

* It is a collection of homogeneous and heterogeneous type of data , separated by comma

* The boundary condition --> (ele1, ele2, ele3,---)

* It can have duplicate values

* It is ordered (it will return the o/p in the same order)

* Length ---> len(variable name) ----> return number of elements present inside a tuple

* It is also immutable data type (original value will not change)

* indexing and slicing can be done on tuple

*HOMOGENEOUS (SAME DATA TYPE ELEMENTS)

```
a  = ("hello", "hoy", "hoy", "hello" , "good")
a
```

Prachu Gouda

"hello", "hoy", "hoy", "hello" , "good"

*HETEROGENEOUS (DIFFERENT DATA TYPE ELEMENTS)


```
t = (12, 42.6, True, "hello",[1, 2, 3])
t
12, 42.6, True, "hello",[1, 2, 3]
```

WAYS TO CREATE TUPLE DATA TYPE:

---

1) DIRECT ASSIGNING:

```
a  = (12, 42, "hello", 75.23)
```

2). BY USING CLASS OBJECT

```
a = tuple((1, 2, 3))
a
(1, 2, 3)
```

Prabhu Gouda

```
#How to create an empty tuple-->??
a=()
a
()


#How to create an empty tuple by using
constructor-->??

b=tuple()
b
()


#How to create a single value tuple-->??

c=(1,)
c
(1,)



# TUPLE METHODS:
----------------------------------------

Finding attributes in tuple data type

dir(tuple)

1.count()
```

Prabhu Gouda

2.index()

1.COUNT()
---
* It will return the number of Occurence of the elements

* If the specified value is not there it will return 0

* syntax : var_name.count(element)

```
t = (45, 42, 78, 12, 42, 45, 12, 45, 75)
#varname. count(element)

t.count(42)
2
t.count(12)
2
t.count(45)
3
t.count(100)
0
```

2.INDEX()
---
* Index will return the Index value of the specified element

Prabhu Gouda

* It will always return the index number of 1st occurrence of specified value

* If the values are not present it will return value error

* Syntax : var_name.index(element)

```
t = (45, 42, 78, 12, 42, 45, 12, 45, 75)
t.index(45)
0
t.index(12)
3

t.index(12,4,9)
6

t.index(45,3,8)
5

t.index(45, 7, 9)
7

t.index(100)

    ValueError: tuple.index(x): x not in tuple
```

eg2:

```
a  = ("hello", "hoy", "hoy", "hello" , "good")
a.index("hoy")
```

Prabhu Gouda

```
1

a.index("hello",1,4)
3
```

INDEXING OF TUPLE:
---

* Fetching / extracting single element is called as indexing

* Syntax : varname[index number]

*+ve indexing start from 0
*-ve indexing start from -1

* len will always start from 1

```
t = ("hello", "hoy", 12.2, 42,"good", 45,[ 12, 45, 75])

t[4]
'good'
t[2]
12.2

t[6][1]
45
t[4][2]
'o'
```

Prachi Gouda

```
#-ve
t[-5]
12.2

t[-1][-2]
45

#SLICING :
-------------------
* To fetch multiple character at a time

* syntax : varname[si: ei: sv]

default value
----------
si = 0
ei = len(var)
sv = 1

t = ("hello", "hoy", 12.2, 42,"good", 45,( 12,
45, 75))

t[1:6:1]
('hoy', 12.2, 42, 'good', 45)

t[0:2]
('hello', 'hoy')


t[-7:-5:1]
```

Prabhu Gouda

```
('hello', 'hoy')

t[0:-5:1]
('hello', 'hoy')

t[-7:2:1]
('hello', 'hoy')

#NOTE:----> IN tuple data type boundary condition
is not mandatory but comma is mandatory(,)
```

LIST DATA TYPE

****

It is a collection of homogeneous and heterogeneous characters

- Homogeneous------> same data type
- Heterogeneous----> combination of characters
- list data type is order
- list data type it will accept duplicate value/elements
- syntax:--> var_name=[ele1,ele2,ele3.....]

Prabhu Gowda

1.How to check the length of the list------>??

- --> by using len function

len(var_name)

a=[1,2,3,4]

len(a)------>4

#Note----> len it will start from ---->1

2.How to create an empty list--->???

var_name=[ ]

3.How to create an empty list by using objects?

var_name=list( )

#indexing:----->

The process of extracting single characters is called as indexing

Prabhu Gouda

syntax:---> var_name[index number/position]

```
#list indexing

#syntax:----> var_name[indexpostion/number]

a=[12,23,45,"hello","python"]
a[2]
45
a[4]
'python'
a[1]
23
a[-5]
12
a[-2]
'hello'
a[34]
Traceback (most recent call last):
  File "<pyshell#57>", line 1, in <module>
    a[34]
IndexError: list index out of range
```

#slicing in list

The process of extracting multiple characters is called slicing

Prabhu Gouda

```
syntax:--> var_name[si:ei+1:sv]


#slicing:---->
#syntax:---> v_n[si:ei:sv]


x=["hello","python","walmart"]
x[1]
'python'
x[1][1:4:1]
'yth'

#mar
x[2][3:6:1]
'mar'

b=["goodmorning"["good night"],"bye"]
Traceback (most recent call last):
  File "<pyshell#70>", line 1, in <module>
    b=["goodmorning"["good night"],"bye"]
TypeError: string indices must be integers, not
'str'




b=["goodmorning",["good night"],"bye"]
b[1]
['good night']
b[1][1][1:4:1]
```

Prasad Gouda

```
Traceback (most recent call last):
  File "<pyshell#73>", line 1, in <module>
    b[1][1][1:4:1]
IndexError: list index out of range

b[1]
['good night']
b[1][1:4:1]
[]
b[1][0][1:4:1]
'ood'



b=["goodmorning",["good night"],"bye"]
b[1]
['good night']
b[1][0]
'good night'
b[1][0][1:4:1]
'ood'

m=[[["python",["sql"],["manual"],["java"]]]]


m[1]
Traceback (most recent call last):
  File "<pyshell#86>", line 1, in <module>
    m[1]
IndexError: list index out of range
m[0][0][1][0]
```

Prabhu Gouda

```
'sql'

m[0]
[['python', ['sql'], ['manual'], ['java']]]
m[0][0]
['python', ['sql'], ['manual'], ['java']]
```

```
x=[["akash","rahul"]]
x[0]
['akash', 'rahul']
x[0][0]
'akash'

x=[[["akash","rahul"]]]
x[0]
[['akash', 'rahul']]
x[0][0]
['akash', 'rahul']
x[0][0][1]
'rahul'
```

```
y=[[[[[[["python"]]]]]]]
y[0][0][0][0][0][0][0]
'python'
```

Prabhu Gouda

```
m=[["hero",[["walmart","instagram"],["facebook"],
["snapchat"],"whatsapp"]]]
m[0][1][0][1][0][1:4:1]
''
m[0]
['hero', [['walmart', 'instagram'], ['facebook'],
['snapchat'], 'whatsapp']]
m[0][0]
'hero'
m[0][1]
[['walmart', 'instagram'], ['facebook'],
['snapchat'], 'whatsapp']
m[0][1][1]
['facebook']
m[0][1][1][0]
'facebook'
m[0][1][1][0][1:4:1]
'ace'
```

```
m=[["hero",[["walmart","instagram"],["facebook"],
["snapchat"],"whatsapp"]]]
m[0]
['hero', [['walmart', 'instagram'], ['facebook'],
['snapchat'], 'whatsapp']]
m[0][1]
[['walmart', 'instagram'], ['facebook'],
['snapchat'], 'whatsapp']
m[0][1][1]
['facebook']
```

Prabhu Gouda

```
m[0][1][1][0]
'facebook'
```

LIST METHODS

```
dir(list)
```

#ADDING ELEMENTS TO THE LIST:

---

1. append()
2. extend()
3. insert()

1.APPEND
---

* It is used add the element to the list

* It will add the element at last position

* we can pass BOTH collection and individual data type

*return None (modifies original list in backside)

Prabhu Gouda

*SYNTAX : var_name.append(elements)

example:

#individual::

```
l = [4, 'active', 'hello']
l.append(45.5)
#return None
l
[4, 'active', 'hello', 45.5]

l.append(7+9j)
#return None
l
[4, 'active', 'hello', 45.5, (7+9j)]
```

2. EXTEND
---
* It is used add the element to the list

* It will EXTEND the existing list, add the element at last position

* we can pass ONLY collection data type

Prabhu Gouda

*return None (modifies original list in backside)

*SYNTAX : var_name.extend(elements)

#extend():---->

#syntax:--> var_name.extend(iterable)

```
a=[1,2,3]
a.extend(45)
Traceback (most recent call last):
  File "<pyshell#176>", line 1, in <module>
    a.extend(45)
TypeError: 'int' object is not iterable

a.extend("python")
a
[1, 2, 3, 'p', 'y', 't', 'h', 'o', 'n']

a.extend([56,89,99])
a
[1, 2, 3, 'p', 'y', 't', 'h', 'o', 'n', 56, 89, 99]

a.extend([[56]])
a
[1, 2, 3, 'p', 'y', 't', 'h', 'o', 'n', 56, 89, 99, [56]]
```

Prabhu Gowda

3.INSERT:
---

* It is used add the element to the list

* It will add the element at SPECIFIED position

* we can pass BOTH collection and individual data type

*return None (modifies original list in backside)

*SYNTAX : var_name.insert(index/position , value)

```
#insert():--->
#syntax:---> var_name.insert(position,value)


n=[12,100,300]
n.insert(1,"python")
n
[12, 'python', 100, 300]

n.insert(3,"java")
n4
Traceback (most recent call last):
  File "<pyshell#195>", line 1, in <module>
    n4
```

Prabhu Gowda

```
NameError: name 'n4' is not defined. Did you
mean: 'n'?
n
[12, 'python', 100, 'java', 300]
4

n
[12, 'python', 100, 'java', 300]

n.insert(1,67+45j)
n
[12, (67+45j), 'python', 100, 'java', 300]

x=3+5j
x
(3+5j)
```

REMOVING ELEMENTS FROM THE LIST

----------3

3 Methods 1 Keyword

1.pop()
2.remove()
3.clear()

Prabhu Gowda

1. POP()
---
* used to remove specific elements from the list
* if you don't specify any index it will remove LAST INDEX VALUE

* if the specified index is not present it Throw INDEX ERROR

* it will return removed value

syntax:--> var_name.pop(position)

#EX:

```
l = ["hello", "good", "morning" , "people"]
l.pop()
'people'
l
['hello', 'good', 'morning']

l.pop(1)
'good'

l
['hello', 'morning']
```

#nested

Prabhu Gouda

```
l = ["hello", "good",[99, 88, "morning" ,
"people"]]

l[2].pop(0)
99
l
['hello', 'good', [88, 'morning', 'people']]


l.pop(5)
Traceback (most recent call last):
  File "<pyshell#20>", line 1, in <module>
    l.pop(5)
IndexError: pop index out of range
```

2. REMOVE()
---

* used to remove specific VALUE from the list

* if you don't specify any index it will Throw TYPE ERROR (we have at least 1 argument)

* if the specified VALUE is not present it Throw VALUEERROR
* it will return None (modifies the original list)

syntax:--> var_name.remove(element)

```
EX:::
------

l = ["hello", "good", "morning" , "people"]
l.remove()
    TypeError: list.remove() takes exactly one
argument (0 given)


l.remove("morning")
#return None
l
['hello', 'good', 'people']

l.remove("afternoon")

    ValueError: list.remove(x): x not in list

#nested

l = ["hello", "good",[99, 88, "morning" ,
"people"]]
l[2].remove(88)
#return None
l
['hello', 'good', [99, 'morning', 'people']]
```

Prabhu Gouda

3. CLEAR()
---

* It will clear all the elements inside the list

* return empty list

*SYNTAX : var_name.clear()

```
l1 = ['hello', 'good', 'people']
l1.clear()
#return None
l1
[]
```

4. DEL KEYWORD
---
* it will remove the entire list from the memory location

*SYNTAX : del var_name

```
l1 = ['hello', 'good', 'people']
del l1
#return None
l1
```

NameError: name 'l1' is not defined. Did you mean: 'l'?

Prabhu Gouda

```
l = ["hello", "good",[99, 88, "morning" ,
"people"]]
del l
#return None
l
    NameError: name 'l' is not defined
```

COUNT()

---

* It returns the count of specified element., which is present inside a list

* If the specified element is not present inside list , it will return 0

* syntax: list.count("element")

#examples

```
l = ["bagalkot",
"davanagere","banglore","hassan","telangana","jai
pur","banglore", "davanagere","banglore"]

l.count("hassan")
1
```

Prabhu Gouda

```
l.count("jaipur")
1

l.count("davanagere")
2

l.count("telangana")
1

l.count("banglore")
3

l.count("japan")
0

l.count("banglore",3)

    TypeError: list.count() takes exactly one
argument (2 given)
```

```
#INDEX():
------------ ------------
* It will return the Index number / position of
the specified element, which is present inside a
list

* It will return the index number of first
Occurrence of the specified element
```

Prachu Gouda

* If the specified value is not there it will return ValueError

*syntax :

```
list.index("ele") / var_name.index("ele")
```

#EG:

```
-----------
l = [75, 45, 95, 75,15, 25,45,75]
l.index(15)
4

l.index(25)
5

l.index(45)
1
l.index(45,3)
6

l.index(75)
0
l.index(75,2)
3
l.index(75,5)
7

l.index(1000)
ValueError: 1000 is not in list
```

Prabhu Gouda

#COPY()
---
* It is used to copy the value present in one variable to another variable

* Copy operation can be performed in 3 ways
1.normal copy
2.shallow copy
3.deepcopy

1.normal copy()
---
*address of original list and copied list will be same

*address of original nested list ad copied nested list will be same

#syntax:----> new var_name=original var_name

```
a=[1,2,3,4]
b=a
a
[1, 2, 3, 4]
b
[1, 2, 3, 4]
```

Prachi Gouda

```
id(a)
2122252969088
id(b)
2122252969088
```

note:
---

*If address is same if we do any changes in any 1 of the variable it will affect the other variable

**If address is different if we do any changes in any 1 of the variable it will not affect the other variable

*SHALLOW COPY:
---
*we will be using inbuilt method copy()
* SYNTAX: new_var = original var .copy()

*address of original list and copied list will be different
*address of original nested list and copied nested list will be same

#examples

```
#syntax:--->
new_var_name=original_var_name.copy()

x=[11,12,13]
y=x.copy()

x
[11, 12, 13]
y
[11, 12, 13]

id(x)
2122258334592
id(y)
2122258483520

#nested shallow copy()

b=[11,12,13,["a","b","c"]]

v=b.copy()

b
[11, 12, 13, ['a', 'b', 'c']]
v
[11, 12, 13, ['a', 'b', 'c']]

id(b)
2122213072896

id(v)
```

Prabhu Gouda

```
2122258482176

id(b[3])
2122258416704

id(v[3])
2122258416704

b
[11, 12, 13, ['a', 'b', 'c']]
b[2]=33
b
[11, 12, 33, ['a', 'b', 'c']]
v
[11, 12, 13, ['a', 'b', 'c']]

v[1]="d"
v
[11, 'd', 13, ['a', 'b', 'c']]

b
[11, 12, 33, ['a', 'b', 'c']]

b[3]
['a', 'b', 'c']
b[3][0]
'a'
b[3][0]=77
b
[11, 12, 33, [77, 'b', 'c']]
v
```

Prabhu Gouda

```
[11, 'd', 13, [77, 'b', 'c']]

b
[11, 12, 33, [77, 'b', 'c']]
v
[11, 'd', 13, [77, 'b', 'c']]

id(b[0])
140714677298280
id(v[0])
140714677298280
```

#DEEPCOPY()
---

*Deepcopy() is a method present inside copy module

* we have to import it

*syntax:
*from copy import deep copy

```
new_var = deepcopy(original variable)
```

*address of original list and copied list will be different

Prabhu Gouda

```
*address of original nested list and copied
nested list will be different

#examples::
---------------------

#step1--->
from copy import deepcopy

#syntax:---> var_name=deepcopy(original_var_name)

m=[1,2,3,4]
n=deepcopy(m)
m
[1, 2, 3, 4]
n
[1, 2, 3, 4]

id(m)
2122258185408
id(n)
2122258483328

#nested deep copy()

c=[11,12,13,["hello","bye"]]
x=deepcopy(c)
c
[11, 12, 13, ['hello', 'bye']]
x
[11, 12, 13, ['hello', 'bye']]
```

Prabhu Gouda

```
id(c)
2122258451136
id(x)
2122258481344

id(c[3])
2122258484096
id(x[3])
2122258457920

c
[11, 12, 13, ['hello', 'bye']]
c[0]=88
c
[88, 12, 13, ['hello', 'bye']]
x
[11, 12, 13, ['hello', 'bye']]

c[3][0]="sql"
c
[88, 12, 13, ['sql', 'bye']]
x
[11, 12, 13, ['hello', 'bye']]

id(c[1])
140714677298312
id(x[1])
140714677298312
```

Prabhu Gouda

SORT()
---

* sort will sort the list

* by default sort will sort the list in ascending order

* in order to sort the list , the list should be homogeneous

* it modifies the original list itself

*SYNTAX : var_name.sort(key = None, reverse = False)

Note: If we want to sort the list in descending order we have to change REVERSE = False to REVERSE = True

#ord()
#it will take 1 argument in the form of string
it will return ascii number

```
ord("A")
65
ord("B")
66
ord("Z")
90
```

Fabri Couda

```
ord("a")
97
ord("z")
122

ord("0")
48
ord("9")
57


#range ord numbers
#0 - 9  = 48 - 57
#A - Z = 65 -90
#a - z = 97 - 122

#remaining numbers will be special characters

#chr()
##it will take 1 argument in the form of number
# it will return value


chr(97)
'a'

chr(122)
'z'

chr(67)
```

Prabhu Gouda

```
'C'

chr(90)
'Z'

chr(48)
'0'

chr(57)
'9'

a=["mankind","apple","mango","crow","cow","dog","
zibra", ]

a.sort()
# based on ascii values
#return None
a
['apple', 'cow', 'crow', 'dog', 'mango',
'mankind', 'zibra']


#reverse parameter
--------------------------
a.sort(reverse = True)
#return None
a
['zibra', 'mankind', 'mango', 'dog', 'crow',
'cow', 'apple']
```

```
key parameter
---------------------
a=["mankind","dog","apple","zibra"
,"mango","crow","cow"]
a.sort(key = len)                                #only
len not ascii values based on 1st element
#return None
a
['dog', 'cow', 'crow', 'apple', 'zibra', 'mango',
'mankind']

a.sort(key = len,reverse = True)
#return None
a
['mankind', 'apple', 'zibra', 'mango', 'crow',
'dog', 'cow']

l = [75, 45, 86, 12, 100, 5]
l.sort()
#RN

l
[5, 12, 45, 75, 86, 100]


l.sort(reverse = True)
#RN
l
[100, 86, 75, 45, 12, 5]

    TypeError: object of type 'int' has no len()
```

Prabhu Gouda

S E T - D A T A T Y P E :

* Set is a mutable data type (changes the original value)

* Set is a mutable data type , But the elements inside a set are immutable data type (all individual , string, tuple)

* Set is unordered (we cannot predict the o/p)

* Set cannot have duplicates

* length of set :--> len(variable)-->
  Note : in len(set) --> it will never consider duplicate values

```
s ={42, 42, 75, 65, 32}
len(s)
4
```

* boundary : {1, 2, 3}

* empty set : set()

* we cannot do INDEXING AND SLICING

NOTE:
immutable ---> hashable --->(all individual , string, tuple)
mutable ------> unhashable --->(list, set , dictionary)

Prabhu Gouda

#WAYS TO CREATE SET DATA TYPE:

1.DIRECT ASSIGNING

```
s = {1, 2, "hell" , "heaven", (1,2,3)}
s
{1, 2, "hell" , "heaven", (1,2,3)}     # we cannot predict o/p
```

2. BY USING Object:

```
s = set()

a = set({1, 2, 3})
a
{1, 2, 3}
```

3. EMPTY SET

```
s = set()
s
set()
type(s) ---> <class 'set'>
```

Prabhu Gouda

#wrong::::::::
s = {}
s
{}
type(s) ---><class 'dict'>

ATTRIBUTES:::::: OF SET
---

dir(data type)

dir(set)

['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_difference_update', 'union', 'update']

INDEXING AND SLICING NOT POSSIBLE

Prabhu Gouda

s = {1, 2, 3, "hello", "hoy"}

s[3]

TypeError: 'set' object is not subscriptable

s[0]

TypeError: 'set' object is not subscriptable

s[1:4:1]

TypeError: 'set' object is not subscriptable

METHODS IN SET :

METHODS IN SET :

1.UNION
---

* It returns a set that contains all the items from the original set and all the items from specified set

Prashu Gouda

* It will create its own set not effect on original and specified set

* SYNTAX : set1.union(set2)

#eg:

Q1]
s = {1, 2, 3}
s1 = {7, 5, 9, 6}

s.union(s1) #s --> original/current set , #s1 --> specified
{1, 2, 3, 5, 6, 7, 9}

s
{1, 2, 3}
s1
{9, 5, 6, 7}

Q2]

original = {45, 66, 33, 99}
specified = {12, 48, 34, 66, 99}
original.union(specified)
{33, 66, 99, 34, 12, 45, 48}

2.UPDATE( )

Prabhu Gouda

---

* It returns a set that contains all the items from the original set and all the items from specified set

* It will effect on original set (all the element from specified set gets add to original set because of immutability)

* original value changes and specified value will not change

* SYNTAX : set1.update(set2)

#eg::

```
s = {1, 2, 3}
s1 = {7, 5, 9, 6}

s.update(s1)

s
{1, 2, 3, 5, 6, 7, 9}
s1
{9, 5, 6, 7}
```

2]

```
s = {1, 2, 3}
s1 = {7, 5, 9, 6}
s1.update(s)
```

Prabhu Gowda

```
s1
{1, 2, 3, 5, 6, 7, 9}
s
{1, 2, 3}
```

OR

update( )

Here it will accept all collection data type but not individual data type

syntax:---> var_name.update(iterable)

```
a={1,2,3}
a.update("hello")
a
{1, 2, 3, 'e', 'o', 'h', 'l'}

a.update(["super"])
a
{1, 2, 3, 'e', 'o', 'super', 'h', 'l'}
```

3.INTERSECTION():

Prabhu Gouda

* It returns a set that contains similarity b/w two or more sets (same values)

* It will create its own set not effect on original and specified set

* SYNTAX : set1.intersection(set2)

#eg :

```
a = {42, 51, 36}
b = {36, 42, 19}
a.intersection(b)
{42, 36}

a
{42, 51, 36}
b
{42, 19, 36}


a = {42, 51, 36}
b = {36, 42, 19}
c = {19, 100, 42}

a.intersection(b , c)
{42}

a
{42, 51, 36}
```

Prabhu Gouda

```
b
{42, 19, 36}
c
{42, 19, 100}
```

#INTERSECTION_UPDATE():
---

* It returns a set that contains similarity b/w two or more sets (same values)

* It will effect on original set (all the element from specified set gets add to original set because of immutability)

* original value changes and specified value will not change

* SYNTAX : set1.intersection_update(set2)

#EG:

```
a = {42, 51, 36}
b = {36, 42, 19}
b.intersection_update(a)


a
{42, 51, 36}
```

```
b
{42, 36}
#--> only common values

c = {19, 100, 42}
a = {42, 51, 36}
b = {36, 42, 19}

c.intersection_update(b, a)

c
{42}

a
{42, 51, 36}

b
{42, 19, 36}
```

7.SYMMETRIC_DIFFERENCE():
---

* It returns a set that contains all the items which are not common among all the set (different value values)

Prabhu Gouda

* It will create its own set not effect on original and specified set

* SYNTAX : set1.symmetric_differece(set2)

#EG:

```
a = {100, 200, 300, 400, 500}
b = {400, 600, 700, 200, 300}

a.symmetric_difference(b)
{700, 100, 500, 600}
b.symmetric_difference(a)
{700, 100, 500, 600}
```

Q1]

```
a = {(1,2,3), 75.2, 96+3j, "hello"}
b = {11, "morning", 75.2, 96 + 3j}

b.symmetric_difference(a)
{(1, 2, 3), 11, 'morning', 'hello'}

a.symmetric_difference(b)
{11, (1, 2, 3), 'morning', 'hello'}

a
{(1,2,3), 75.2, 96+3j, "hello"}
b
{11, "morning", 75.2, 96 + 3j}
```

Prashant Gouda

Q2]

```
a = {(1, 2, 3), "hello" ,66}
b = {"HELLO", (4, 5, 6), 66}
a.symmetric_difference(b)
{(1, 2, 3), (4, 5, 6), 'hello', 'HELLO'}
b.symmetric_difference(a)
{(1, 2, 3), (4, 5, 6), 'hello', 'HELLO'}

a
{(1, 2, 3), "hello" ,66}

b
{"HELLO", (4, 5, 6), 66}
```

Q3]

```
a = {44, 55, 22}
b = {55, 12, 14}
c = {12, 16, 55}
b.symmetric_difference(a, c)

    TypeError: set.symmetric_difference() takes
exactly one argument (2 given)
```

#8.SYMMETRIC_DIFFERENCE_UPDATE():
---

* It updates a base/ original set with the items that are not common among all the set (different values)

Prachu Gouda

* It will effect on original set (all the element from specified set gets add to original set because of immutability)

* original value changes and specified value will not change

* SYNTAX : set1.symmetric_difference_update(set2)

#EG:

Q1]
```
a = {44, 55, 22}
b = {22,11,33}

a.symmetric_difference_update(b)
#return None
a
{33, 11, 44, 55}
b
{33, 11, 22}
```

Q2]
```
a = {44, 55, 22}
b = {100, 75, 95, 55}
b.symmetric_difference_update(a)
```

Prabhu Gowda

```
#return None
b
{100, 75, 44, 22, 95}
a
{44, 22, 55}
```

```
#ADDING ELEMENTS TO THE SET :   1 METHOD
-------------------------------------------------------------------
```

*ADD():
---

* It adds an item , to the existing set

* It will add the element randomly

* We can pass both collection(string and tuple) and individual data type

*If the item already exists, the add() method does not add the element

* SYNTAX : var_name.add(immutable data type)

```
a = {"facebook", "instagram", "snapchat"}
a.add("whatsapp")
#return None
```

```
a
{'instagram', 'snapchat', 'facebook', 'whatsapp'}

a.add("whatsapp")
#return None
a
{'instagram', 'snapchat', 'facebook', 'whatsapp'}

#no set cannot have duplicate values

a.add(1)
#return None
a
{1, 'instagram', 'whatsapp', 'snapchat',
'facebook'}

>>
a.add(4,"hello")

    TypeError: set.add() takes exactly one
argument (2 given)

>>
a.add((4,"hello"))
a
{1, 'instagram', 'whatsapp', (4, 'hello'),
'snapchat', 'facebook'}
>>
a.add({4,5})

    TypeError: unhashable type: 'set'
```

Prashu Couda

```
#we should pass only immutable type of data

>>
a.add((7,8,"whatsapp"))
a
{1, 'instagram', 'whatsapp', (4, 'hello'), (7, 8,
'whatsapp'), 'snapchat', 'facebook'}

##################################################
################
#REMOVING METHODS : -
-----------------------------------
1.remove()

2.discard()

3.pop()

1.DISCARD():
---------------------

* It removes the specified item from set

* If the item is not present it will not raise
any error  (X)

*SYNTAX :  varname.discard(item)

s = {1, 'instagram', 'whatsapp', (4, 'hello'),
(7, 8, 'whatsapp'), 'snapchat', 'facebook'}
```

Prabhu Gowda

```
s.discard((4, "hELlo"))
#not raise any error
s
{(7, 8, 'whatsapp'), 1, 'instagram', 'whatsapp',
(4, 'hello'), 'facebook', 'snapchat'}

s.discard((4, "hello"))
s
{(7, 8, 'whatsapp'), 1, 'instagram', 'whatsapp',
'facebook', 'snapchat'}

s.discard("zoo")
#not raise any errors
{(7, 8, 'whatsapp'), 1, 'instagram', 'whatsapp',
'facebook', 'snapchat'}

s.discard("snapchat")
s
{(7, 8, 'whatsapp'), 1, 'instagram', 'whatsapp',
'facebook'}

s.discard(1)
s
{(7, 8, 'whatsapp'), 'instagram', 'whatsapp',
'facebook'}
```

2.REMOVE():

Prachu Gouda

* It removes the specified item from set

* If the item is not present it will raise KEY ERROR

*SYNTAX : varname.remove(item)

```
#EX:
------

s = {1, 'instagram', 'whatsapp', (4, 'hello'),
(7, 8, 'whatsapp'), 'snapchat', 'facebook'}

s.remove((7,8,"whatsapp"))
#return None
s
{1, 'instagram', 'whatsapp', (4, 'hello'),
'facebook', 'snapchat'}

>>
s.remove("facebook")
s
{1, 'instagram', 'whatsapp', (4, 'hello'),
'snapchat'}

>>
s.remove("INSTAGRAM")
    KeyError: 'INSTAGRAM'

>>
s.remove("instagram")
```

Prabhu Gowda

```
#return None
s
{1, 'whatsapp', (4, 'hello'), 'snapchat'}
```

3.POP()

---

* It will remove Random element from the set

* It will 0 argument

* It will return removed element

* If the set becomes empty it will raise KEYERROR

* SYNTAX: varname.pop( )

#EX:

```
se = {1, 75.6, 9+7j, False, "king", "cool", (7,
8, "hye")}
se.pop()
False

se.pop("cool")
Traceback (most recent call last):
  File "<pyshell#78>", line 1, in <module>
    se.pop("cool")
TypeError: set.pop() takes no arguments (1 given)
```

Prabhu Gowda

```
se.pop()
1

se.pop()
'king'

se.pop()
(7, 8, 'hye')

se.pop()
'cool'

se.pop()
75.6

se.pop()
(9+7j)

se.pop()
Traceback (most recent call last):
  File "<pyshell#92>", line 1, in <module>
    se.pop()
KeyError: 'pop from an empty set'
```

Prabhu Gouda

1.DIFFERENCE():
---

* It returns a set that contains non similar value b/w two or more sets (different value values)

* It will create its own set not effect on original and specified set

* SYNTAX : set1.difference(set2)

#eg:

```
a = {75, 95, 35, 15}
b = {15, 25, 65, 75}
a.difference(b)
{35, 95}
b.difference(a)
{65, 25}

a
{75, 95, 35, 15}
b
{15, 25, 65, 75}

a = {"hello", "hai", 144, 75.96}
b = {144, "lion", "tiger", "hai", 99.9}
a.difference(b)
{'hello', 75.96}
b.difference(a)
{'tiger', 99.9, 'lion'}
```

Prabhu Gouda

```
a
{"hello", "hai", 144, 75.96}
b
{144, "lion", "tiger", "hai", 99.9}

a = {75, 95, 35, 15}
b = {15, 25, 65, 75}
c = {78, 24, 75, 95}
a.difference(b, c)
{35}

a
{75, 95, 35, 15}
b
{15, 25, 65, 75}
c
 {78, 24, 75, 95}
```

#2.DIFFERENCE_UPDATE():
---

* It returns a set that contains difference b/w two or more sets (different values)

* It will effect on original set (all the element from specified set gets add to original set because of immutability)

Prabhu Gouda

* original value changes and specified value will not change

* SYNTAX : set1.difference_update(set2)

#eg:-

```
q1.
a = {75, 95, 35, 15}
b = {15, 25, 65, 75}
a.difference_update(b)
#return None

a
{35, 95}
b
{65, 25, 75, 15}

q2.
a = {"hello", "hai", 144, 75.96}
b = {144, "lion", "tiger", "hai", 99.9}
b.difference_update(a)
#return None

b
{99.9, 'lion', 'tiger'}

a
{'hello', 144, 75.96, 'hai'}
```

Prabhu Gouda

```
#without different value

a = {1, 2, 3}
b = {1, 2, 3}
a.difference(b)
set()
b.difference(a)
set()
```

```
#BOOLEAN METHODS OF SET
-----------------------------------------------
* They return True and False

1.isdisjoint()

2.issuperset()

3.issubset()

1.isdisjoint():
-----------------
* It returns True if none of the items are
present in both the sets otherwise False
*Syntax : set1.isdisjoint(set2)

eg:
---
Q1]
```

Prabhu Gouda

```
a = {1, 2, 3}
b = {4, 5, 6}
a.isdisjoint(b)
True

b.isdisjoint(a)
True


Q2]
a = {89, 42, 66}
b = {49, 82, 66}

a.isdisjoint(b)
False

Q3]
a = {"hello", "hai" , (1, 2, 3)}
b = {"Hello", "haI" , (44, 26, 35)}
a.isdisjoint(b)
True

b.isdisjoint(a)
True

--------------------------------------------------
-------
2.issuperset():
-----------------
```

Prachu Gouda

* It returns True if all the items in reference set exists in the original set, otherwise False’

* Syntax : set1.issuperset(set2)

Q1]

```
a = { 111, 222, 333, 444}
b = { 111, 666, 222, 555, 333, 444}

# b--> original ,  a --> reference/specified

b.issuperset(a)
True

a.issuperset(b)
False
```

Q2]

```
a = {"good", "morning", "guys", "how", "are",
"you"}
b = {"are", "good", "guys", "you"}

a.issuperset(b)
True

b.issuperset(a)
False
```

---

3. issubset()

Prabhu Gouda

---

* It returns True if all the items in original set exists in the reference set, otherwise False

* Syntax : set1.issubset(set2)

Q1]

```
a = { 111, 222, 333, 444}
b = { 111, 666, 222, 555, 333, 444}
b.issubset(a)
False
# b--> original ,  a --> reference/specified

a.issubset(b)
True
```

Q2]

```
a = {"good", "morning", "guys", "how", "are",
"you"}
b = {"are", "good", "guys", "you"}
b.issubset(a)
True
a.issubset(b)
False
```

Prabhu Gouda

DICTIONARY DATA TYPE

* It is a collection of key - value pair

* Each element associated with unique key (duplicate keys not allowed)

* Each Key and value pairs are separated by , (comma)

* Boundary :--> {key: value}

* Length of dict --> len(dict) --> return the count of keys present in dictionary

* It cannot be indexed or sliced

* It is mutable data type

* Values are dependent on Keys

CHARACTERISTICS OF DICTIONARY : -
---

1. Key cannot be duplicate

2. Key will be single element

3. Key should be immutable data type

Prabhu Gouda

4. Values can be any data type

5. Values can be accessed with the help keys

WAYS TO CREATE DICTIONARY :-

---

1. DIRECT ASSIGNING VALUES:

```
d = {"hello" : 10}
d
{"hello" : 10}

d1 = {1 : [1, 2, 3]}
d1
{1 : [1, 2, 3]}
```

2.BY USING CLASS OBJECT:

```
d = dict({"a": 10})
d
{'a': 10}
```

3.EMPTY SET

```
1.d = {}
```

Prabhu Gouda

```
2.d = dict()


#COMPOSITE KEYS:
-----------------------------
* If we want to create multiple keys we can use
composite keys

* we can pass multiple keys in a tuple that is
nothing but composite keys

#EXAMPLE:
----------------------
Q1.
d = {"rajdeep", "singh" : 1}
SyntaxError: invalid syntax

d = {("rajdeep", "singh" ): 1}
d
{('rajdeep', 'singh'): 1}


Q2.
d1 = {"kamalesh ","kumar" : 3, "kamalesh",
"rajkumar" :10}
SyntaxError: invalid syntax

d1 = {("kamalesh ","kumar") : 3, ("kamalesh",
"rajkumar" ):10}

d1
```

Prachi Gouda

```
{('kamalesh ', 'kumar'): 3, ('kamalesh',
'rajkumar'): 10}
```

ACCESSING VALUES FROM DICTIONARY::::

1. key - value syntax

2. get()

1. key - value syntax
---
* It will give the value of a specified key

* If the key which is specified is not present inside the dictionary it will raise KEYERROR

* syntax : dict[key]

 ----> ex :-- a["rajdeep"]

```
a = {"narmada" : 1 , "shalini": 2, "ashwini" : 3,
"rajdeep":4}
a["shalini"]
2

a["narmada"]
1

a["rajdeep"]
```

Prabhu Gouda

```
4

a["ashwini"]
3

a["Ashwini"]
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    a["Ashwini"]
KeyError: 'Ashwini'
```

* DUE TO UPPER CASE A

2. get():
---

* It returns the values of the item with specified key

* If the key is not present, then it will give the second parameter as values.

* SYNTAX : dict.get(key , parameter2/default value)

#EXAMPLE:--

```
universe = {"sky" : 12.1 , 2 : "moon" , 45.2 : "sun" }
universe.get("sky")
```

Prabhu Gouda

12.1

```
universe.get("Sky")
# NO ERROR

universe.get("Sky", " SPECIFIED KEY IS NOT
PRESENT")
' SPECIFIED KEY IS NOT PRESENT'


universe.(10, "Check The Key which you have
passed")
SyntaxError: invalid syntax

universe.get(10, "Check The Key which you have
passed")
'Check The Key which you have passed'

universe.get(45.2, "pass the proper key")
'sun'
```

NOTE : i will take only 2 parameters

```
universe.get(45.2, 2, "pass the proper key")
Traceback (most recent call last):
  File "<pyshell#30>", line 1, in <module>
    universe.get(45.2, 2, "pass the proper key")
TypeError: get expected at most 2 arguments, got
3
```

Prabhu Gouda

ACCESSING ONLY KEYS ONLY VALUE ONLY ITEM

1.KEYS()

---

* It returns the list of the all keys present in a dictionary

* List will reflect any changes done to the dictionary

*SYNTAX : var_name. keys()

```
food = {"idli" : 25, "dosa" : 45, "upma" :20,
"poori" :40, "vada" : 15}

food.keys()
dict_keys(['idli', 'dosa', 'upma', 'poori',
'vada'])

food["puliyogare"] = 30
food
{'idli': 25, 'dosa': 45, 'upma': 20, 'poori': 40,
'vada': 15, 'puliyogare': 30}

food.keys()
dict_keys(['idli', 'dosa', 'upma', 'poori',
'vada', 'puliyogare'])
```

Prabhu Gowda

2.VALUES()

* It returns a list of all the values present in dictionary
* SYNTAX: var_name.values()

```
#EXAMPLES:
-------------------

food.values()
dict_values([25, 45, 20, 40, 15, 30])

food["idli"]25
SyntaxError: incomplete input

food["idli"]                    #ACCESSING THE
VALUES
25

food["idli"] = 22          # ACCESS AND UPDATE THE
VALUE
food
{'idli': 22, 'dosa': 45, 'upma': 20, 'poori': 40,
'vada': 15, 'puliyogare': 30}

food.values()
dict_values([22, 45, 20, 40, 15, 30])
```

3.ITEMS()

```
* It returns a list of the  KEY : VALUE pair
present in dictionary
* List will reflect any changes done to the
dictionary
*SYNTAX :  var_name. items()
---> (key: value)


#EXAMPLES:
-------------------

food = {'idli': 22, 'dosa': 45, 'upma': 20,
'poori': 40, 'vada': 15, 'puliyogare': 30}

food.items()
dict_items([('idli', 22), ('dosa', 45), ('upma',
20), ('poori', 40), ('vada', 15), ('puliyogare',
30)])

food["upma"]
20

food["upma"] = 18

food
{'idli': 22, 'dosa': 45, 'upma': 18, 'poori': 40,
'vada': 15, 'puliyogare': 30}

food.items()
```

Prabhu Gowda

```
dict_items([('idli', 22), ('dosa', 45), ('upma',
18), ('poori', 40), ('vada', 15), ('puliyogare',
30)])


# TO GET THE ALL ATTRIBUTES IN DICTIONARY
-------------------------------------------------------
-------------------
# dir(data type)

dir(dict)
['__class__', '__class_getitem__',
'__contains__', '__delattr__', '__delitem__',
'__dir__', '__doc__', '__eq__', '__format__',
'__ge__', '__getattribute__', '__getitem__',
'__getstate__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__ior__', '__iter__',
'__le__', '__len__', '__lt__', '__ne__',
'__new__', '__or__', '__reduce__',
'__reduce_ex__', '__repr__', '__reversed__',
'__ror__', '__setattr__', '__setitem__',
'__sizeof__', '__str__', '__subclasshook__',
'clear', 'copy', 'fromkeys', 'get', 'items',
'keys', 'pop', 'popitem', 'setdefault', 'update',
'values']


#ADDING ELEMENTS TO A DICTIONARY:
-------------------------------------------------------
----------
1. key -value syntax
```

2. update()
3. set default
4. from keys

1. key -value syntax:

```
#key - value syntax

#dict[new key ] = value

fam = {"davanagere" : "glass house", "mysore" :
"palace" }

fam["jaipur"] = "hawa mahal"
fam
{'davanagere': 'glass house', 'mysore': 'palace',
'jaipur': 'hawa mahal'}

fam["salem"] = "mango"
fam
{'davanagere': 'glass house', 'mysore': 'palace',
'jaipur': 'hawa mehal', 'salem': 'mango'}
```

2. update()
---

2 syntax :
---

dict.update({key :value})

update examples:--

Prachi Gouda

```
a={1:2}
a.update({"a":9})
a
{1: 2, 'a': 9}
a.update({"b":89,"c":"d"})
a
{1: 2, 'a': 9, 'b': 89, 'c': 'd'}
a.update({(90,67):[1,2,3]})
a
{1: 2, 'a': 9, 'b': 89, 'c': 'd', (90, 67): [1,
2, 3]}
```

3. set default:

---

* It returns the value of the item with specified key, if the value is not specified it adds None as its value

* If the key does not exist , it will insert the key, with specified value

#EXAMPLE:

#if the value and key passed at a time

```
p = {'banglore': 576001, 'kalaburagi': 585101,
'jaipur': 302001}
```

Prabhu Gouda

```
p.setdefault("dawanagere", 577004)
577004
p
{'banglore': 576001, 'kalaburagi': 585101,
'jaipur': 302001, 'dawanagere': 577004}

#if only key passed  not values
-----------------------------------------------
p.setdefault("tamilnadu")
p
{'banglore': 576001, 'kalaburagi': 585101,
'jaipur': 302001, 'dawanagere': 577004, 'tamil
nadu': None}


Q2]
p = {"banglore" : 576001}
p.setdefault("kalaburagi")
p
{'banglore': 576001, 'kalaburagi': None}
p.setdefault("jaipur")
p
{'banglore': 576001, 'kalaburagi': None,
'jaipur': None}


# we cannot update with set default we can give
only keys , by default it will assign the value
as None
```

```
# if you want to replace that None value use 1st
2 methods (key -value syntax or update())

#key - value pair

p["kalaburagi"] = 585101
p
{'banglore': 576001, 'kalaburagi': 585101,
'jaipur': None}

#update()

p.update({"jaipur" : 302001})
p
{'banglore': 576001, 'kalaburagi': 585101,
'jaipur': 302001}

#4. FROMKEYS():
------------------------
* It returns a dictionary with specific keys and
the specified value

* keys are mandatory. An iterable specifying key
of the new dictionary

* value are optional : The for all the keys will
be NONE by default

* we can add the value as well as we can update
the value by using fromkeys
iterable : all iterable data type
```

```
#syntax:--->
dict.fromkeys(iterable/variable_name)
a="hello"
dict.fromkeys(a)
{'h': None, 'e': None, 'l': None, 'o': None}
dict.fromkeys(a,7)
{'h': 7, 'e': 7, 'l': 7, 'o': 7}

l=[1,2,3,5]
dict.fromkeys(l)
{1: None, 2: None, 3: None, 5: None}
dict.fromkeys(l,"hey")
{1: 'hey', 2: 'hey', 3: 'hey', 5: 'hey'}

c=(45,78,89)
dict.fromkeys(c,"hero")
{45: 'hero', 78: 'hero', 89: 'hero'}
```

REMOVING ELEMENTS FROM A DICTIONARY :

---

1. POP()
2. POPITEM()

1. POP()

---

* It removes the specified item from the dictionary and return the removed value as a output

* if the key is not present it will return the value which you have passed

*SYNTAX : VAR_NAME.POP(key,defaultvalue)

```
#EXAMPLES:
-----------------------

d = {"virat" : 18, "msd" : 7, "rohit" : 45, "abd"
: 17}
d.pop("abd")
17

d
{'virat': 18, 'msd': 7, 'rohit': 45}

d.pop("abd","already removed")
'already removed'


d.pop("virat", "key not present")
18

d
{'msd': 7, 'rohit': 45}
```

Pinchu Couda

#POP ITEM:
---

* It removes the item that was inserted at the last into the dictionary

* It returns the removed key value pair as tuple (k , v)

* When the dict becomes empty if we use this method it will raise KEYERROR

*SYNTAX : VAR_NAME.POPITEM()

#EXAMPLES :-

```
d = {"virat" : 18, "msd" : 7, "rohit" : 45, "abd"
: 17}

d.popitem()
('abd', 17)

d
{'virat': 18, 'msd': 7, 'rohit': 45}

d.popitem()
('rohit', 45)

d
{'virat': 18, 'msd': 7}
```

Prabhu Gouda

```
d.popitem()
('msd', 7)

d
{'virat': 18}

d.popitem()
('virat', 18)

d
{}

d.popitem()
Traceback (most recent call last):
  File "<pyshell#74>", line 1, in <module>
    d.popitem()
KeyError: 'popitem(): dictionary is empty'
```

OPERATORS

```
#OPERATOR

#defn : These are simple which is used to perform
specific operations on the operand


#Python operators are used to perform some
operations/task on variables and values
```

Prabhu Gouda

#Types of Operators

#1.Arithmetic operator

#2.logical operator

#3.relational operator

#4.Bitwise operator

#5.assignment operator

#6.membership operator

#7.identity operator

Arithmetic operator Types:

#1. Additional operator

#2. subtraction operator

#3. Multiplication operator

Prabhu Gouda

```
#4. True/Division operator


#5. Floor division operator
#6. Power operator

#Additional Operator
a = 45
b = 65
a + b
110

a = 4.5
b = 5.5
a + b
10.0
a = 78
b = 98
a + b
176

a = 3+5j
b = 8+4j
a + b
(11+9j)

a = 8+0j
b = 5j
a + b
```

Prabhu Gouda

```
(8+5j)

a = 8+0j
b = 5
a + b
(13+0j)

a = 6+3j
b = 5.7
a + b
(11.7+3j)

a = True
b = False
a + b
1

a = True
b = 2+3j
a + b
(3+3j)

3.765 + (2+5j) + True + 7j
(6.765000000000001+12j)
```

Prabhu Gouda

```
#Concatenation operator(+)
#If we use '+' operator with collection data
types we call it as Concatenation
```

```
a = 'hai'
b = 'hello'
a + b
'haihello'

s = " "

a = 'hello'
b = 'world'
a + s + b
'hello world'

a = 'i am'
b = 'developer'
c = 'software'
d = 'tester'

a + s + c + b
'i am softwaredeveloper'

a + s + c + s + b
'i am software developer'

a + s + c + s + d
'i am software tester'

a = [1, 2, 3, 4, 5]
b = [2, 3, 6, 8, 10]
a + b
[1, 2, 3, 4, 5, 2, 3, 6, 8, 10]
```

Prabhu Gouda

```
a = (1, 2, 3, 4, 5)
b = (3, 5, 7, 9, 12)

a + b
(1, 2, 3, 4, 5, 3, 5, 7, 9, 12)


a = {1, 2, 3, 4, 5}
b = {2, 4, 6, 8, 10}
a + b
Traceback (most recent call last):
  File "<pyshell#92>", line 1, in <module>
    a + b
TypeError: unsupported operand type(s) for +:
'set' and 'set'


a = {'a': 'apple', 'b': 'banana'}
b = {'c': 'chocloate', 'd': 'donut'}
a + b
Traceback (most recent call last):
  File "<pyshell#96>", line 1, in <module>
    a + b
TypeError: unsupported operand type(s) for +:
'dict' and 'dict'


c = {a ** b}
Traceback (most recent call last):
  File "<pyshell#97>", line 1, in <module>
```

```
    c = {a ** b}
TypeError: unsupported operand type(s) for ** or
pow(): 'dict' and 'dict'



a | b
{'a': 'apple', 'b': 'banana', 'c': 'chocolate',
'd': 'donut'}
c = {**a, **b}
c
{'a': 'apple', 'b': 'banana', 'c': 'chocolate',
'd': 'donut'}



#note : Whenever we perform concatenation both
operands should be of same datatype
#* Concatenation can't be performed on set and
dictionary

#Subtraction operator(-) :
a = 56
b = 34
a - b
22

b - a
-22

a = 56.878
```

Prabhat Gupta

```
b = 65.78
a - b
-8.902000000000001

a = 3+5j
b = 2+6j
a - b
(1-1j)

a = 5+8j
b = 3+19j
b - a
(-2+11j)

a = True
b = False
a - b
1
a - a
0

'hello' - 'hai'
Traceback (most recent call last):
  File "<pyshell#129>", line 1, in <module>
    'hello' - 'hai'
TypeError: unsupported operand type(s) for -:
'str' and 'str'

[1, 2, 3] - [4, 5, 6]
Traceback (most recent call last):
  File "<pyshell#131>", line 1, in <module>
```

Prabhu Gouda

```
    [1, 2, 3] - [4, 5, 6]
TypeError: unsupported operand type(s) for -:
'list' and 'list'

(1, 2, 3) - (4, 5, 6)
Traceback (most recent call last):
  File "<pyshell#133>", line 1, in <module>
    (1, 2, 3) - (4, 5, 6)
TypeError: unsupported operand type(s) for -:
'tuple' and 'tuple'

{1, 2, 3, 4} - {2, 4, 6, 8, 10}
{1, 3}

a = {'a': 'apple', 'b': 'banana'}
b = {'c': 'chocloate', 'd': 'donut'}
a - b
Traceback (most recent call last):
  File "<pyshell#139>", line 1, in <module>
    a - b
TypeError: unsupported operand type(s) for -:
'dict' and 'dict'

#note : The values given on operands should be
single value data type, apart from that we can
use it with set.
```

```
#Multiplication(*)
#It is used to find the product of two numeric
values

#whenever multiplication operator is used with
single value DT, It behaves like a multiplication
operator and result is product

#if the same multiplication operator is used in
collection DT i,e str, list, tuple it behaves
like a replication operator

a = 2 * 4
a
8

a = 34
b = 23
a * b
782

a = 2.34
b = 1.54
a * b
3.6035999999999997

a = 4+3j
b = 3+2j
a * b
(6+17j)
```

Prabhu Gowda

```
a = True
b = False
a * b
0
```

```
#replication operator
a = 'hai'
a * 4
'haihaihaihai'

a = [1, 2, 3]
a * 2
[1, 2, 3, 1, 2, 3]

a = (1, 2, 3)
a * 3
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```

```
#True Division operator

#defn : True division operator provides the
actual value after dividing two values
```

Prachu Gouda

```
#True division can be performed only on Single
value DataType

#If we do True division any number by zero we
results in 'ZeroDivisionerror'

a = 56
b = 5
a / b
11.2

a = 45.98
b = 4.5
a / b
10.217777777777776

a = 5+6j
b = 2+3j
a / b
(2.1538461538461537-0.2307692307692307 8j)

a = True
b = False
a / b
Traceback (most recent call last):
  File "<pyshell#202>", line 1, in <module>
    a / b
ZeroDivisionError: division by zero
```

Prachu Gouda

```
#floor Division(//)--> it is used to get only
quotient

#defn : It is used to return floor value after
dividing two numericals

#it can be performed only on int, float, bool

a = 46
b = 7
a // b
6

a = 76.98
b = 4.2
a // b
18.0

a = 67
b = 9
a // b
7

34 // 6
5

76 // 3
25
```

Prabhu Gouda

```
#modulus operator(%)

#defn : Modulus operator is used to return
remainder after dividing 2 values

#it can be used only with int, float, bool

15 % 2
1

45 % 3
0

67 % 7
4

98 % 4
2

65.4 % 6.5
0.4000000000000057

45 % 23
22
```

Prabhu Gowda

```
#power operator(**):

#defn : It is used to find the power of a given
value

#it can be used only with single value DT

2 ** 2   # 2^2
4

2 ** 3
8

15 * 2
30
15 ** 2
225

21 ** 2
441

6 ** True     # 6^1
6

7 ** False
1
```

Prabhu Gowda

Logical Operators

```
#Logical operators are used to compare the
condition statements

#there are 3 logical operators

#              *AND, *OR, *NOT

#AND operator: It returns True only when both
arguments are true, else False

a = 5
b = 3
a < 10 and b < 5
True

a < 10 and b < 2
False

a < 4 and b < 4
False

True and True
True
```

Prabhu Gouda

#OR operator : It returns True if at least one argument is true else False

```
a = 10
b = 5
a > b or b < a
True

a < b or b < a
True

a > b or b > a
True

b > a or a < b
False
```

#NOT operator : This NOT operator is always going to return the reverse complement

```
not True
False

not !
SyntaxError: invalid syntax
not 1
False

not False
```

Prabhu Gouda

```
True

not 0
True
```

Membership Operator:

```
#defn : This is type of operator which is used to
check whether the given data is a part of the
specified group items


#It will return True if the specified data is
present or else it returns False


#types of Membership operators :
              * in operator
              * not in operator


#In operator : This operator is used to check
whether if the given data is present/part of the
specified group data items
```

```
#It will return True if specified data is present
else it will return False

18 in [10, 20, 90, 40, 18, 28, 35]
True

19 in [10, 20, 90, 40, 18, 28, 35]
False

'10' in [10, 20, 90, 40, 18, 28, 35]
False

'hello' in ['hello', 'hi', 'python', 'c', 'helo']
True

0.123 in [0.12, 0.43, 9.98]
False

0.123 in [0.123, 0.43, 9.98]
True

18 in {10, 20, 90, 40, 18, 28, 35}
True
```

#NOT IN operator : This operator is used to check whether if the given data is not present/part of the specified group data items

Prabhu Gouda

```
#It will return True if specified data is not
present else it will return False.

18 not in [10, 20, 90, 40, 18, 28, 35]
False

19 not in [10, 20, 90, 40, 18, 28, 35]
True

'10' not in [10, 20, 90, 40, 18, 28, 35]
True

'hello' not in ['hello', 'hi', 'python', 'c',
'helo']
False

0.123 not in [0.12, 0.43, 9.98]
True
```

Identity operator :

```
#two types of identity operator

#* is operator  * is not operator
```

is operator : It returns 'True' only when two reference variables pointing to the same object.

Prabhu Gouda

```
num1 = 10
num2 = 20
num3 = 10
id(num1)
2003664175632
id(num2)
2003664175952
id(num3)
2003664175632


num1 is num2
False
num1 is num3
True
num3 is num2
False

msg1 = 'python;
SyntaxError: unterminated string literal
(detected at line 1)
msg1 = 'python'
msg2 = 'pyhton'
msg3 = 'python'

id(msg1)
2003704462320
id(msg2)
2003705292976
id(msg3)
```

Prabhu Gouda

```
2003704462320

msg1 is msg2
False

msg1 is msg3
True

msg3 is msg2
False

lst1 = [10, 20, 30]
lst2 = [20, 30, 40]
lst3 = [10, 20, 30]
lst4 = [20, 30, 40]

id(lst1)
2003705455680
id(lst2)
2003705456128
id(lst3)
2003705455872
id(lst4)
2003705455360

lst1 is lst3
False

lst2 is lst4
False
```

Prabhu Gouda

is not operator : It returns True only when two reference variabes are not pointed to the same object, else it will return False.

```
num1 = 10
num2 = 20
num3 = 10

id(num1)
2003664175632
id(num2)
2003664175952
id(num3)
2003664175632

num1 is not num3
False

num1 is not num2
True

num2 is not num3
True

msg1 = 'python'
msg2 = 'pyhton'
msg3 = 'python'

id(msg1)
2003704462320
id(msg2)
```

Prabhu Gouda

```
2003705292976
id(msg3)
2003704462320

msg1 is not msg3
False

msg2 is not msg1
True

msg2 is not msg3
True
```

ASSIGNMENT OPERATOR: (=)

```
#defn : Assignment operators are those which are
used to assign the values to the variables in
python.

a = 5
a
5
a += 5        #a = a + 5
a
10
a = 10
a += 20            # a = a + 20
a
30
```

Prabhu Gouda

```
a = 15
a -= 5              #a  = a - 5
a
10

a = 10
a *= 2          #a = a * 2
a
20

#note : Every arithmetic operators combines with
assignment operators

#i,e ----> +=, -=, *=, /=, %=, //=, **=, &=, |=,
^=, <<=, >>=

#Python does not support increment and decrement
operators and the

addition and subtraction are just a sign(symbol)
not an increment and decrement operators
True == False
False

3+2j == 3+2j
True

'cat' == 'cat'
True
```

Prabhu Gouda

```
[1, 2, 3] == [1, 2, 3]
True

{1, 2, 3} == {1, 2, 3}
True

{'name': 'alex', 'age': 23} == {'name': 'alex',
'age': 23}
True
```

Bitwise operator

```
#Bitwise operator : Bitwise operators are those
which operate on 1 or more bits or binary numbers
at their individual bits.

#there are 6 bitwise operators i,e

#1. Bitwise AND (&)

#2. Bitwise OR (|)

#3. Bitwise XOR (^)

#4. Bitwise NOT/Compliment (~ tilde)

#5. Bitwise left shift operator (<<)

#6. Bitwise right shift operator (>>)
```

Prachi Gouda

#Bitwise AND operator (&) : In Bitwise operator if both the bits are 1 then it returns 1 else 0

Truth Table

op1	op2	res
1	0	0
1	1	1
0	0	0
0	1	0

#if we want to get binary number of a digit --->
bin()

```
53
53
bin(53)
'0b110101'
bin(58)
'0b111010'

53 & 58
48

a = 34
b = 43
bin(a)
'0b100010'
bin(b)
```

Prabhu Gouda

```
'0b101011'
a & b
34

a = 89
b = 56
bin(a)
'0b1011001'
bin(b)
'0b111000'
a & b
24
```

```
#Bitwise OR operator (|) : In Bitwise OR operator
if atleast one bit is 1 then result is 1, else
result is 0

Truth Table

op1   op2     res
1     0         1
1     1         1
0     0         0
0     1         1
```

```
a = 53
b = 58
```

Prabhat Gouda

```
bin(a)
'0b110101'
bin(b)
'0b111010'
a | b
63

a = 34
b = 43
bin(a)
'0b100010'
bin(b)
'0b101011'
a | b
43
```

#Bitwise XOR operator(^) : In bitwise XOR operator if both bits are different then result is 1, else result is 0

Truth Table

op1	op2	res
1	0	1
1	1	0
0	0	0
0	1	1

Prabhu Gouda

```
a = 53
b = 58
bin(a)
'0b110101'
bin(b)
'0b111010'
a ^ b
15

a = 34
b = 43
bin(a)
'0b100010'
bin(b)
'0b101011'
a ^ b
9

#Bitwise NOT/COMPLEMENT operator( ~) : The most
significant bits acts as the sign bit,
#syntax : n = -(n+1)
a = 34
~ a
-35

b = 56
~ b
-57

c = 45
~ c
```

Prabhu Gouda

```
-46

#Bitwise left shift operator (<<) :
10 << 2
40

bin(25)
'0b11001'
25 << 3
200


#Bitwise left shift operator (<<) : In this the
left operand value will be shifted to the L.H.S
by the number of bits specified b/w right operand


#Bitwise right shift operator(>>) :In this the
value will be losing the bit by the number of
bits specified with the right operand.

10 >> 2
2

25 >> 3
3
```

Relational operators

#Relational operators

#1. lesser than operator(<)

#2. greater than operator(>)

#3. lesser than equals (<=)

#4. greater than equals(>=)

#defn : Relation operators are used to compare 2 values in order to check relation b/w them.

#*. relational operators will always results in boolean values

#*. Whenever we are going to compare a single value DT we will compare at once.

#*whenever we are going to compare Collection DT we will be comparing value by value

Prabhu Gowda

#*If both the values of both operands are the same then we go for the next value.

#This process keeps on continuing until we find different values

#*if all the values are the same then we will compare its length.

#*Whenever we compare two Collection DT both of them should belong to the same DT.

lesser than operator(<):

#lesser than operator returns True if operand 1 is lesser than operand 2, else False

```
2 < 1
False

True < False
False

5 < 4
False
```

Prachu Gouda

```
5.4 < 5.5
True

'one' < 'two'
True

#ascii values
ord('o')
111
ord('t')
116

'five' < 'four'
True
ord('f')
102
ord('f')
102
ord('i')
105
ord('o')
111

#Ascii values ranges
#A-Z --> 65 - 90
#a -z ---> 97- 122
#0 - 9 --> 48 - 57

'seven' < 'one'
False
```

Prabhu Gouda

```
ord('s')
115
ord('o')
111

'hai' < 'hai'
False

'hai' < 'haii'
True

[1, 2, 3] < [2, 3, 4]
True

[3, 4, 5] < [1, 2, 3, 4]
False

[4, 5, 6] < [4, 5, 6, 7]
True

[5, 6, 7] < [5, 6, 7]
False

{1, 2, 3} < {3, 2, 1}
False
```

Greater than operator(>): It returns True if operand 1 is greater than operand 2 else, False.

```
5 > 2
True
```

Prabhu Gouda

```
6 > 7
False


46 > 23
True

45.67 > 34.87
True

'lion' > 'tiger'
False
ord('l')
108
ord('t')
116

[2, 3, 4] > [4, 5, 6]
False

['hai', 'hello'] > ['hello', 'hai']
False

(5, 4, 3) > (6, 5, 4)
False

(7, 6, 4) > (8)
Traceback (most recent call last):
  File "<pyshell#82>", line 1, in <module>
```

Prachu Gouda

```
    (7, 6, 4) > (8)
TypeError: '>' not supported between instances of
'tuple' and 'int'
(7, 6, 4) > (8,)
False

{2, 3, 4} > {4, 3, 2}
False

#lesser than equals(<=) : It will return True if
operand 1 is lesser than or equals to operand 2
else False.
45 <= 50
True

46 <= 23
False

45 <= 45
True

65 <= 67
True

'hai' <= 'hello'
True
ord('a')
97
ord('e')
101
ord('hai')
```

Prabhu Gowda

```
Traceback (most recent call last):
  File "<pyshell#99>", line 1, in <module>
    ord('hai')
TypeError: ord() expected a character, but string
of length 3 found

'hai' <= 'hai'
True

[1, 2, 3] <= [2, 3, 4]
True

[3, 4, 5] <= [3, 4, 5]
True

(2, 3, 4) <= (4, 5, 6)
True

(5, 6, 7) <= (5, 6, 7)
True

{4, 5, 6} <= {4, 5, 6}
True
```

Greater than equals(>=) : It returns True if operand 1 is greater than or equals to operand 2 else, False.

```
35 >= 98
False
```

Prabhu Gouda

```
36.445 >= 36.445
True

'word' >= 'letter
SyntaxError: unterminated string literal
(detected at line 1)
'word' >= 'letter'
True
ord('w')
119
ord('l')
108

'sentence' >= 'alphabets'
True
ord('s')
115
ord('a')
97

[10, 86, 98] >= [10, 86, 99]
False
```

#note : We cannot compare greater than equals to Set and Dictionary.

#Equals to operator(=) : It returns True if operand 1 is equals to operand 2 else False

Prabhu Gouda

```
#Equals to operator(==) : It returns True if
operand 1 is equals to operand 2 else False
25 == 35
False
```

```
#not equals to (!=) : It returns True if operand
1 is not equals to operand 2, else False.
45 != 43
True

45 != 45
False

'sunset' != 'sunset'
False

{10, 20, 30} != {30, 20, 10}
False

['hello', 'python'] != ['hello

SyntaxError: unterminated string literal
(detected at line 1)
['hello', 'python'] != ['hello', 'java']
```

Prabhu Gowda

```
True

5 != 2+3j

True
```

Prabhu Gouda